

Date : __/__/__



CÂU HỎI & TRẢ LỜI

★ 100+ CÂU HỎI QUAN TRỌNG ★

CÂU HỎI PHÒNG VẤN PYTHON

Từ Cơ Bản đến Nâng Cao



</> Code.
📖 Học.
✏️ Luyện tập.
🐛 Debug.
🎯 Thành công.

★ BẠN SẼ NHẬN ĐƯỢC GÌ ★



100+ CÂU HỎI

Những câu hỏi phỏng vấn Python quan trọng nhất kèm đáp án rõ ràng.



BAO QUÁT MỌI CHỦ ĐỀ

Từ cơ bản đến nâng cao – mọi thứ bạn cần trong một tài liệu.



TẬP TRUNG PHÒNG VẤN

Đáp án được trình bày rõ ràng, giúp bạn tự tin hơn.



PHÙ HỢP MỌI TRÌNH ĐỘ

Người mới bắt đầu, người đang học & người đi làm – tài liệu này dành cho bạn!



"Càng luyện tập nhiều, bạn càng giỏi hơn."





1. PYTHON BASICS + CÚ PHÁP

Date : __/__/__



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1 Python là gì?

Trả lời:

Python là một ngôn ngữ lập trình bậc cao (high-level), thông dịch (interpreted) và đa năng (general-purpose). Nó được thiết kế để dễ đọc và dễ sử dụng.

2 Các đặc điểm chính của Python là gì?

Trả lời:

- Đơn giản và dễ học
- Ngôn ngữ thông dịch (interpreted)
- Kiểu dữ liệu động (dynamically typed)
- Hướng đối tượng (object-oriented)
- Có thể chạy đa nền tảng (portable / cross-platform)
- Thư viện chuẩn (standard library) phong phú
- Mã nguồn mở, được cộng đồng hỗ trợ mạnh mẽ

3 Python có phân biệt chữ hoa/chữ thường không?

Trả lời:

Có, Python phân biệt chữ hoa/chữ thường (case-sensitive). Ví dụ, `Name`, `name` và `NAME` là ba định danh (identifier) khác nhau.

```
Name = "Alice"
name = "Bob"
print(Name)    # Alice
print(name)    # Bob
```

4 Thụt lề (indentation) trong Python là gì?

Trả lời:

Thụt lề dùng để xác định một khối lệnh (code block) trong Python. Đây là yêu cầu bắt buộc, và Python không dùng dấu ngoặc nhọn {} như các ngôn ngữ khác.

```
if True:
    print("This is inside the block")
    print("Indentation matters")
print("This is outside the block")
```

5 Phần mở rộng của file Python là gì?

Trả lời:

Phần mở rộng của file Python là `.py`

Ví dụ: `program.py`

6 Làm thế nào để in kết quả ra màn hình trong Python?

Trả lời:

Chúng ta dùng hàm `print()` để hiển thị kết quả.

```
print("Hello, World!")
print(123)
print(10.5)
```

7 Làm thế nào để nhận input trong Python?

Trả lời:

Chúng ta dùng hàm `input()` để nhận dữ liệu từ người dùng. Hàm này luôn trả về dữ liệu ở dạng string.

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print("Name:", name)
print("Age:", age)
```

8 Python có yêu cầu khai báo kiểu dữ liệu cho biến không?

Trả lời:

Không, Python có kiểu dữ liệu động (dynamically typed). Bạn không cần khai báo kiểu cho biến, kiểu dữ liệu sẽ được tự động suy ra.

```
x = 10          # int
x = "Hello"     # str
x = 3.14        # float
```

9 Ép kiểu (type casting) là gì?

Trả lời:

Ép kiểu (type casting) là quá trình chuyển đổi một kiểu dữ liệu này sang kiểu dữ liệu khác.

```
x = "10"
y = int(x)    # string to integer
z = float(y)  # integer to float
print(type(y)) # <class 'int'>
print(type(z)) # <class 'float'>
```

10 Ký hiệu nào được dùng để viết comment trong Python?

Trả lời:

Ký hiệu `#` được dùng cho comment một dòng (single-line). Python không hỗ trợ comment nhiều dòng bằng `/* */` như trong C/Java.

```
# This is a single-line comment
print("Hello")
```

QUICK FACTS

- ☒ Python là ngôn ngữ thông dịch, dễ đọc
- ☒ Không cần khai báo kiểu dữ liệu
- ☒ Thụt lề (indentation) là bắt buộc
- ☒ Dùng `#` cho comment một dòng

KEY TAKEAWAY

- ☒ Nắm chắc cú pháp cơ bản là nền tảng để học sâu hơn về Python
- ☒ Luyện tập viết code mỗi ngày để ghi nhớ cú pháp



2. VARIABLES + KIỂU DỮ LIỆU

Date : _/ _/ _



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1 Biến (variable) trong Python là gì?

Trả lời:

Biến là một "vật chứa" (container) lưu trữ một giá trị, và giá trị này có thể thay đổi sau này.

```
x = 10          # integer
name = "Alice"  # string
pi = 3.14       # float
```

2 Chúng ta có cần khai báo biến trong Python không?

Trả lời:

Không, Python không có lệnh khai báo biến riêng. Biến được tạo ra ngay khi bạn gán giá trị cho nó.

```
x = 5          # variable x is created
x = 10         # value of x is updated
```

3 Các quy tắc đặt tên biến là gì?

Trả lời:

- Phải bắt đầu bằng một chữ cái (a-z, A-Z) hoặc dấu gạch dưới (_)
- Có thể chứa chữ cái, chữ số và dấu gạch dưới
- Không được bắt đầu bằng chữ số
- Phân biệt chữ hoa/chữ thường (age, Age và AGE là khác nhau)
- Không được dùng các từ khóa (keyword) của Python

```
valid_name = 100
_temp2 = 3.5
Name1 = "John"
```

4 Tên biến có phân biệt chữ hoa/chữ thường không?

Trả lời:

Có, Python phân biệt chữ hoa/chữ thường.

```
name = "Alice"
Name = "Bob"
print(name)  # Alice
print(Name)  # Bob
```

5 Làm thế nào để xóa một biến?

Trả lời:

Chúng ta dùng từ khóa `del` để xóa một biến.

```
x = 100
del x
print(x)  # NameError: name 'x' is not defined
```

6 Sự khác nhau giữa `=` và `==` là gì?

Trả lời:

`=` dùng để gán giá trị (assignment). `==` dùng để so sánh giá trị (comparison).

```
x = 10          # gán giá trị
y = 10
print(x == y)   # so sánh -> True
```

7 Kiểu dữ liệu động (dynamic typing) trong Python là gì?

Trả lời:

Python có kiểu dữ liệu động. Nó tự động xác định kiểu dữ liệu của một biến.

```
x = 10          # int
x = "Hello"     # now x becomes str
x = 10.5        # now x becomes float
```

8 Các kiểu dữ liệu cơ bản trong Python là gì?

Trả lời:

Các kiểu dữ liệu cơ bản trong Python gồm: `int`, `float`, `str`, `bool`, `list`, `tuple`, `set`, `dict`.

Kiểu	Mô tả	Ví dụ
int	Số nguyên	10, -5, 1000
float	Số thực (thập phân)	3.14, -0.001, 2.5e3
str	Văn bản / Chuỗi (String)	"Hello", 'Python'
bool	Boolean (True / False)	True, False
list	Tập hợp có thứ tự, mutable	[1, 2, 3]
tuple	Tập hợp có thứ tự, immutable	(1, 2, 3)
set	Tập hợp không thứ tự, duy nhất	{1, 2, 3}
dict	Cặp Key-Value (mapping)	{'a': 1, 'b': 2}

9 Làm thế nào để kiểm tra kiểu dữ liệu của một biến?

Trả lời:

Chúng ta dùng hàm `type()`.

```
x = 3.14
print(type(x))  # <class 'float'>
```

10 Một biến có thể lưu nhiều kiểu dữ liệu khác nhau không?

Trả lời:

Có, nhờ vào tính năng kiểu dữ liệu động (dynamic typing).

```
x = "123"       # string
x = 123         # integer
x = [1, 2, 3]   # list
```

QUICK FACTS

- Python có kiểu dữ liệu động
- Không cần khai báo kiểu biến
- Tên biến phân biệt chữ hoa/thường
- Dùng `del` để xóa biến
- Dùng `type()` để kiểm tra kiểu dữ liệu

COMMON TYPE CHECKS

```
isinstance(10, int)  # True
isinstance(3.5, float)  # True
isinstance("Hi", str)  # True
isinstance(True, bool)  # True
isinstance([1,2], list)  # True
```

KEY TAKEAWAY

- Biến dùng để lưu trữ dữ liệu
- Không cần khai báo trước
- Python có kiểu dữ liệu động
- Có nhiều kiểu dữ liệu dựng sẵn
- Hiểu rõ kiểu dữ liệu để dùng đúng cách!

→ ĐẶT TÊN BIẾN TỐT GIÚP CODE DỄ ĐỌC VÀ DỄ BẢO TRÌ. →



3. STRINGS + PHƯƠNG THỨC CHUỖI

Date: ____/____/____



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1 Chuỗi (string) trong Python là gì?

Trả lời:

Chuỗi là một dãy ký tự (sequence of characters) được đặt trong dấu nháy đơn, nháy kép hoặc ba dấu nháy.

```
s1 = 'Hello'      # single quotes
s2 = "Hello"     # double quotes
s3 = '''Hello'''  # triple quotes
```

2 Làm thế nào để truy cập các ký tự trong chuỗi?

Trả lời:

Chúng ta dùng chỉ số (indexing). Chỉ số bắt đầu từ 0.

```
s = "Python"
print(s[0]) # P
print(s[3]) # h
print(s[-1]) # n
print(s[-2]) # o
```

Index:	0	1	2	3	4	5
	P	y	t	h	o	n
	-6	-5	-4	-3	-2	-1

3 Làm thế nào để cắt chuỗi (slice)?

Trả lời:

Chúng ta dùng cú pháp slicing: `s[start:end:step]`

```
s = "Python Programming"
print(s[0:6]) # Python
print(s[7:]) # Programming
print(s[:6]) # Python
print(s[::2]) # Pto rgann
print(s[::-1]) # PnohtyP
```

4 Làm thế nào để kiểm tra một chuỗi có chứa chuỗi con không?

Trả lời:

Dùng từ khóa `in`.

```
s = "I Love Python"
print("Python" in s) # True
print("Java" in s) # False
```

5 Làm thế nào để đổi kiểu chữ hoa/thường của chuỗi?

Trả lời:

```
s = "PyThOn"
print(s.lower()) # python
print(s.upper()) # PYTHON
print(s.capitalize()) # Python
print(s.title()) # Python
print(s.swapcase()) # pYtHoN
```

6 Làm thế nào để nối (concatenate/join) các chuỗi?

Trả lời:

Chúng ta dùng toán tử `+` hoặc phương thức `join()`.

```
a = "Hello"
b = "World"
c = a + " " + b # Hello World
d = "-".join(["Python", "is", "fun"]) # Python-is-fun
```

7 Làm thế nào để tìm độ dài của một chuỗi?

Trả lời:

Dùng hàm `len()`.

```
s = "Hello, Python!"
print(len(s)) # 14
```

8 Một số phương thức chuỗi (string method) thường dùng?

Trả lời:

Phương thức	Mô tả	Ví dụ
lower()	Chuyển thành chữ thường	"HELLO".lower() -> "hello"
upper()	Chuyển thành chữ hoa	"hello".upper() -> "HELLO"
strip()	Xóa khoảng trắng đầu/cuối	"hi ".strip() -> "hi"
replace()	Thay thế chuỗi con	"a-b".replace("-", "_") -> "a_b"
split()	Tách chuỗi thành list	"a,b".split(',') -> ['a', 'b']
join()	Nối list thành chuỗi	"-".join(['a', 'b']) -> "a-b"
startswith()	Kiểm tra tiền tố	"python".startswith('py') -> True
endswith()	Kiểm tra hậu tố	"file.txt".endswith('.txt') -> True
find()	Trả về vị trí chuỗi con	"hello".find('e') -> 1
count()	Đếm số lần xuất hiện	"banana".count('a') -> 3

9 Làm thế nào để định dạng chuỗi (format string) trong Python?

Trả lời:

Chúng ta có thể dùng f-strings, phương thức `format()` hoặc toán tử `%`.

```
name = "Alice"
age = 25
# f-string (Python 3.6+)
print(f"My name is {name} and age is {age}")
# format() method
print("My name is {} and age is {}".format(name, age))
# % operator
print("My name is %s and age is %d" % (name, age))
```

10 Làm thế nào để xóa khoảng trắng trong chuỗi?

Trả lời:

```
s = " Python is awesome "
print(s.strip()) # Python is awesome
print(s.replace(" ", "")) # Pythonisawesome
print("".join(s.split())) # Pythonisawesome
```



4. LIST + TUPLE + SET



Date : __/__/____

10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1 List trong Python là gì?

Trả lời:

List là một tập hợp có thứ tự (ordered), có thể thay đổi (mutable) và cho phép các giá trị trùng lặp.

```
lst = [10, 20, 30, 'hello', 40.5]
print(type(lst)) # <class 'list'>
print(lst[0]) # 10
print(lst[-1]) # 40.5
```

2 Làm thế nào để truy cập các phần tử trong List?

Trả lời:

Chúng ta dùng indexing và slicing.

```
lst = [10, 20, 30, 40, 50]
print(lst[0]) # 10 (phần tử đầu)
print(lst[-1]) # 50 (phần tử cuối)
print(lst[1:4]) # [20, 30, 40]
print(lst[:3]) # [10, 20, 30]
print(lst[::2]) # [10, 30, 50]
```

3 Một số phương thức (method) List thường dùng?

Trả lời:

Method	Mô tả
append(x)	Thêm một phần tử vào cuối
extend(iterable)	Thêm tất cả phần tử từ một iterable
insert(i, x)	Chèn x vào vị trí i
remove(x)	Xóa lần xuất hiện đầu tiên của x
pop(i)	Xóa & trả về phần tử tại vị trí i
clear()	Xóa tất cả phần tử
index(x)	Trả về vị trí xuất hiện đầu tiên của x
count(x)	Đếm số lần xuất hiện của x
sort()	Sắp xếp list theo thứ tự tăng dần
reverse()	Đảo ngược thứ tự các phần tử

4 Tuple trong Python là gì?

Trả lời:

Tuple là một tập hợp có thứ tự, không thể thay đổi (immutable) và cho phép giá trị trùng lặp.

```
tup = (10, 20, 'hello', 30.5)
print(type(tup)) # <class 'tuple'>
print(tup[0]) # 10
print(tup[-1]) # 30.5
```

Lưu ý: Chúng ta không thể thay đổi phần tử của tuple.
tup[0] = 100 ❌ Error

5 Tuple khác List như thế nào?

Trả lời:

- List có thể thay đổi (mutable), tuple thì không (immutable)
- List dùng ngoặc vuông [], tuple dùng ngoặc tròn ()
- List chậm hơn tuple
- Tuple được dùng khi dữ liệu cố định

List	Tuple
Mutable	Immutable
Dùng []	Dùng ()
Chậm hơn	Nhanh hơn
Tốn nhiều bộ nhớ hơn	Tốn ít bộ nhớ hơn

6 Một số phương thức tuple thường dùng?

Trả lời:

Method	Mô tả
count(x)	Đếm số lần x xuất hiện
index(x)	Trả về vị trí xuất hiện đầu tiên của x
len(tup)	Trả về độ dài tuple
max(tup)	Trả về phần tử lớn nhất
min(tup)	Trả về phần tử nhỏ nhất
tup1 + tup2	Nối hai tuple
tup * n	Lặp lại tuple n lần

7 Set trong Python là gì?

Trả lời:

Set là một tập hợp không có thứ tự (unordered), có thể thay đổi (mutable) và không cho phép giá trị trùng lặp.

```
s = {10, 20, 30, 20, 10} # duplicates removed
print(s) # {10, 20, 30}
print(type(s)) # <class 'set'>
```

8 Một số phương thức set thường dùng?

Trả lời:

Method	Mô tả
add(x)	Thêm phần tử x vào set
update(iterable)	Thêm nhiều phần tử
remove(x)	Xóa phần tử x (lỗi nếu không tìm thấy)
discard(x)	Xóa phần tử x (không lỗi nếu không tìm thấy)
pop()	Xóa & trả về một phần tử ngẫu nhiên
clear()	Xóa tất cả phần tử
union(set)	Trả về hợp của hai set
intersection(set)	Trả về phần tử chung
difference(set)	Trả về phần tử không có trong set kia

9 Sự khác nhau giữa List, Tuple và Set?

Trả lời:

Đặc điểm	List	Tuple	Set
Có thứ tự	Yes	Yes	No
Mutable	Yes	No	Yes
Cho phép trùng lặp	Yes	Yes	No
Syntax	[]	()	{}
Indexing	Yes	Yes	No
Trường hợp dùng	Dùng chung	Dữ liệu cố định	Phần tử duy nhất

10 Làm thế nào để chuyển đổi qua lại giữa List, Tuple và Set?

Trả lời:

- List sang Tuple: `t = tuple([1, 2, 3])` # (1, 2, 3)
- Tuple sang List: `l = list((1, 2, 3))` # [1, 2, 3]
- Set sang List: `l = list({1, 2, 3})` # [1, 2, 3] (thứ tự có thể khác)
- List sang Set: `s = set([1, 2, 2, 3])` # {1, 2, 3}



5. DICTIONARIES + COMPREHENSIONS

Date: / /



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

A. DICTIONARIES

1 Dictionary trong Python là gì?

Trả lời:

Dictionary là một tập hợp không có thứ tự (unordered), có thể thay đổi (mutable), lưu trữ các cặp key-value. Key phải là duy nhất và immutable.

```
d = {'name': 'Alice', 'age': 25, 'city': 'Delhi'}
print(type(d)) # <class 'dict'>
```

2 Làm thế nào để truy cập giá trị trong dictionary?

Trả lời:

Chúng ta dùng key để truy cập giá trị.

```
d = {'name': 'Alice', 'age': 25, 'city': 'Delhi'}
print(d['name']) # Alice
print(d.get('age')) # 25
print(d.get('country', 'Not Found')) # Not Found
```

3 Một số phương thức dictionary thường dùng?

Trả lời:

Method	Mô tả
keys()	Trả về tất cả các key
values()	Trả về tất cả các value
items()	Trả về các cặp key-value dạng tuple
update(d)	Cập nhật dictionary bằng một dictionary khác
get(key, default)	Trả về giá trị nếu key tồn tại, ngược lại trả về giá trị mặc định
pop(key)	Xóa key và trả về giá trị của nó
popitem()	Xóa và trả về phần tử được thêm sau cùng
clear()	Xóa tất cả phần tử
copy()	Trả về một bản sao (shallow copy)

4 Làm thế nào để thêm hoặc cập nhật cặp key-value?

Trả lời:

Chúng ta có thể thêm hoặc cập nhật bằng phép gán.

```
d = {}
d['name'] = 'Alice' # Thêm
d['age'] = 25
d['name'] = 'Alicia' # Cập nhật
print(d) # {'name': 'Alicia', 'age': 25}
```

5 Làm thế nào để xóa phần tử khỏi dictionary?

Trả lời:

Chúng ta dùng `pop()`, `del` hoặc `clear()`.

```
d = {'a': 1, 'b': 2, 'c': 3}
d.pop('b') # Xóa key 'b'
del d['c'] # Xóa key 'c'
# d.clear() # Xóa tất cả
print(d) # {'a': 1}
```

B. COMPREHENSIONS

6 Comprehension trong Python là gì?

Trả lời:

Comprehension cung cấp cách viết ngắn gọn để tạo list, dictionary, set và generator expression.

7 List Comprehension hoạt động như thế nào?

Trả lời:

Tạo một list mới từ một iterable có sẵn.

```
squares = [x**2 for x in range(6)]
print(squares) # [0, 1, 4, 9, 16, 25]
```

```
even = [x for x in range(10) if x % 2 == 0]
print(even) # [0, 2, 4, 6, 8]
```

8 Dictionary Comprehension hoạt động như thế nào?

Trả lời:

Tạo một dictionary mới bằng một expression.

```
d = {x: x**2 for x in range(1, 6)}
print(d) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
d2 = {x: x for x in 'hello'}
print(d2) # {'h': 'h', 'e': 'e', 'l': 'l', 'o': 'o'}
```

9 Set Comprehension hoạt động như thế nào?

Trả lời:

Tạo một set mới từ một iterable có sẵn.

```
squares = {x**2 for x in range(6)}
print(squares) # {0, 1, 4, 9, 16, 25}
```

```
unique_chars = {ch for ch in 'programming'}
print(unique_chars) # {'p', 'r', 'o', 'g', 'a', 'm', 'i', 'n'}
```

10 Generator Expression hoạt động như thế nào?

Trả lời:

Tương tự list comprehension nhưng trả về một generator object (lazy evaluation - tính toán khi cần).

```
gen = (x**2 for x in range(5))
print(type(gen)) # <class 'generator'>
for val in gen:
    print(val, end=' ') # 0 1 4 9 16
```

COMPREHENSION SYNTAX

List: [expression for item in iterable if condition]

Dict: {key: value for item in iterable if condition}

Set: {expression for item in iterable if condition}

Gen: (expression for item in iterable if condition)



6. CONDITIONS + LOOPS



Date : _/ _/ _

10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1 Câu lệnh điều kiện (conditional statement) trong Python là gì?

Trả lời:
Câu lệnh điều kiện được dùng để đưa ra quyết định dựa trên các điều kiện.

```
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

Điều kiện đúng
Điều kiện sai
Điều kiện sai

2 Các toán tử so sánh (comparison operator) trong Python là gì?

Trả lời:

Toán tử	Ý nghĩa	Ví dụ	Kết quả
==	Bằng	5 == 5	True
!=	Khác	5 != 3	True
>	Lớn hơn	5 > 3	True
<	Nhỏ hơn	5 < 3	False
>=	Lớn hơn hoặc bằng	5 >= 5	True
<=	Nhỏ hơn hoặc bằng	5 <= 3	False

3 Toán tử logic (logical operator) trong Python là gì?

Trả lời:

Toán tử	Ý nghĩa	Ví dụ	Kết quả
and	Đúng nếu cả hai đều đúng	(5 > 3 and 2 < 4)	True
or	Đúng nếu có ít nhất một đúng	(5 < 3 or 2 < 4)	True
not	Đảo ngược kết quả	not(5 > 3)	False

4 Cấu trúc if-elif-else là gì?

Trả lời:
Được dùng để kiểm tra nhiều điều kiện.

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Điều kiện 1
Điều kiện 2
Nếu tất cả sai

5 Cấu lệnh if lồng nhau (nested if) là gì?

Trả lời:
Một câu lệnh if nằm bên trong một câu lệnh if khác.

```
x = 10
if x > 0:
    if x % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
```

Dùng khi cần kiểm tra nhiều cấp độ điều kiện.

6 Vòng lặp for trong Python là gì?

Trả lời:
Vòng lặp for dùng để duyệt qua một sequence (list, tuple, string, v.v.).

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

apple, banana, mango

7 Hàm range() là gì?

Trả lời:
Hàm range() tạo ra một dãy số.

```
for i in range(5):
    print(i)
for i in range(1, 6):
    print(i)
for i in range(1, 10, 2):
    print(i)
```

0 1 2 3 4
1 2 3 4 5
1 3 5 7 9

range(start, stop, step)

8 Vòng lặp while trong Python là gì?

Trả lời:
Vòng lặp while lặp lại một khối lệnh chừng nào điều kiện còn đúng (True).

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

1 2 3 4 5

Đừng quên cập nhật biến, nếu không vòng lặp sẽ chạy vô hạn (infinite loop).

9 Sự khác nhau giữa vòng lặp for và while?

Trả lời:

for loop	while loop
Dùng khi biết trước số lần lặp	Dùng khi không biết trước số lần lặp
Duyệt qua một sequence	Lặp lại khi điều kiện còn đúng
Ví dụ: list, string, range	Phù hợp cho vòng lặp không xác định
Phù hợp cho vòng lặp xác định	Cần tự cập nhật biến thủ công

10 break, continue và pass trong Python là gì?

Trả lời:
Đây là các câu lệnh điều khiển vòng lặp (loop control statement).

Lệnh	Mô tả
break	Dùng vòng lặp hoàn toàn
continue	Bỏ qua lần lặp hiện tại, tiếp tục lần tiếp theo
pass	Không làm gì cả. Dùng làm placeholder

```
break
for i in range(5):
    if i==3:
        break
    print(i)
# 0 1 2

continue
for i in range(5):
    if i==2:
        continue
    print(i)
# 0 1 3 4

pass
for i in range(5):
    if i==2:
        pass
    print(i)
# 0 1 3 4
```

QUICK FACTS

- Thụt lề (indentation) rất quan trọng trong Python
- Vòng lặp for dùng khi biết trước sequence
- Vòng lặp while dùng khi chưa biết số lần lặp
- break dùng vòng lặp, continue bỏ qua 1 lần

EXAMPLE CORNER

```
# Kiểm tra số nguyên tố (Prime)
n = 17
is_prime = True
for i in range(2, n):
    if n % i == 0:
        is_prime = False
        break
print(is_prime) # True
```

KEY TAKEAWAY

- Điều kiện giúp chương trình đưa ra quyết định
- Vòng lặp giúp lặp lại công việc tự động
- Hãy chọn đúng loại vòng lặp cho từng bài toán



7. FUNCTIONS + LAMBDA + RECURSION

Date: / /



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1. Function trong Python là gì?

Trả lời:

Function là một khối code có thể tái sử dụng (reusable), thực hiện một nhiệm vụ cụ thể. Nó giúp code có tính module hóa và có thể tái sử dụng.

```
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet("Alice")) # Hello, Alice!
```

2. Các loại function trong Python là gì?

Trả lời:

Có hai loại chính:

- Built-in Functions (hàm dựng sẵn)
- User-defined Functions (hàm do người dùng tự định nghĩa)

Loại	Ví dụ
Built-in	<code>print()</code> , <code>len()</code> , <code>max()</code> , <code>min()</code>
User-defined	<code>def add(a,b): return a+b</code>
Anonymous	lambda arguments: expression
Recursive	Function tự gọi lại chính nó

3. Làm thế nào để định nghĩa một function trong Python?

Trả lời:

Chúng ta dùng từ khóa `def`.

```
def add(a, b):  
    """Function này trả về tổng hai số"""  
    return a + b  
  
result = add(5, 3)  
print(result) # 8
```

Syntax:
`def function_name(parameters):`
statements
return value

4. Parameter và argument của function là gì?

Trả lời:

- Parameter:** Các biến được định nghĩa trong phần khai báo function.
- Argument:** Các giá trị được truyền vào function khi gọi.

```
def multiply(a, b): # a, b là parameter  
    return a * b  
  
ans = multiply(4, 5) # 4, 5 là argument  
print(ans) # 20
```

5. Có những loại argument nào trong Python?

Trả lời:

Python hỗ trợ nhiều loại argument khác nhau.

Loại	Ví dụ	Mô tả
Positional	<code>add(2, 3)</code>	Truyền theo thứ tự
Keyword	<code>add(b=3, a=2)</code>	Truyền theo tên
Default	<code>add(2)</code>	Dùng giá trị mặc định nếu không truyền
Variable-length	<code>add(1,2,3,4)</code>	Nhận số lượng argument bất kỳ (n)

6. Lambda function trong Python là gì?

Trả lời:

Lambda function là một function không tên (anonymous), được định nghĩa bằng từ khóa `lambda`.

```
square = lambda x: x * x  
print(square(5)) # 25  
  
add = lambda a, b: a + b  
print(add(2, 3)) # 5
```

Dùng khi cần một function nhỏ, chỉ dùng trong thời gian ngắn.

7. Lambda function khác function thông thường như thế nào?

Trả lời:

Đặc điểm	Function thường	Lambda
Syntax	Dùng <code>def</code>	Dùng <code>lambda</code>
Tên	Có tên	Không tên (anonymous)
Thân hàm	Nhiều câu lệnh	Chỉ một expression
Return	return là tùy chọn	Return ngầm định
Sử dụng	Mục đích chung	Function ngắn, đơn giản

8. Recursion (đệ quy) trong Python là gì?

Trả lời:

Recursion là kỹ thuật mà một function tự gọi lại chính nó để giải quyết các bài toán con nhỏ hơn.

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
  
print(factorial(5)) # 120
```

Quan trọng: Luôn phải có điều kiện dừng (base condition), nếu không sẽ gây ra đệ quy vô hạn.

9. Điều kiện dừng (base condition) trong đệ quy là gì?

Trả lời:

Base condition là điều kiện giúp dừng các lần gọi đệ quy tiếp theo.

```
def count_down(n):  
    if n == 0: # Base condition  
        print("Done!")  
    else:  
        print(n)  
        count_down(n-1)  
  
count_down(3) # 3 2 1 Done!
```

Nếu không có base condition, function sẽ tự gọi lại mãi mãi và gây ra lỗi.

10. Ưu điểm và nhược điểm của đệ quy (recursion) là gì?

Trả lời:

Ưu điểm:

- Code đơn giản và dễ hiểu
- Hữu ích cho các bài toán có bản chất đệ quy (cây, backtracking, chia để trị)

Nhược điểm:

- Tốn nhiều bộ nhớ hơn (stack space)
- Có thể gây tràn stack (stack overflow) nếu đệ quy quá sâu

QUICK FACTS

- Function giúp tái sử dụng code
- Lambda function chỉ chứa một expression
- Đệ quy luôn cần có base condition
- Argument có nhiều loại: positional, keyword, default...

EXAMPLE CORNER

```
# Fibonacci đệ quy  
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)  
  
print(fib(6)) # 8
```

KEY TAKEAWAY

- Function giúp code module hóa, dễ bảo trì
- Lambda phù hợp cho function ngắn gọn
- Luôn đặt base condition khi dùng đệ quy



8. OOP + CLASSES + INHERITANCE

Date: _/ _/ _



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

1 OOP trong Python là gì?

Trả lời:

OOP (Object Oriented Programming - lập trình hướng đối tượng) là một mô hình lập trình dựa trên khái niệm object và class. Nó giúp xây dựng code có khả năng mở rộng, tái sử dụng và dễ bảo trì.

```
# Tạo một class
class Car:
    def __init__(self, brand, model):
        self.brand = brand # attribute
        self.model = model

    def info(self): # method
        return f"{self.brand} {self.model}"

c = Car("Toyota", "Innova")
print(c.info()) # Toyota Innova
```

2 Class trong Python là gì?

Trả lời:

Class là một khuôn mẫu (blueprint) để tạo ra các object. Nó định nghĩa một tập hợp các attribute và method.

```
class Student:
    def __init__(self, name, age):
        self.name = name # attribute
        self.age = age

    def details(self):
        return f"{self.name} is {self.age} years old"

s = Student("Alice", 20)
print(s.details()) # Alice is 20 years old
```

3 Object trong Python là gì?

Trả lời:

Object là một thực thể (instance) của class. Nó có danh tính, trạng thái và hành vi riêng.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return "Woof!"

d = Dog("Buddy") # object của class Dog
print(d.name) # Buddy
print(d.bark()) # Woof!
```

4 Phương thức __init__() là gì?

Trả lời:

__init__() là một phương thức đặc biệt (constructor) được tự động gọi khi một object được tạo ra.

```
class Student:
    def __init__(self, name, age): # constructor
        self.name = name
        self.age = age

s = Student("Bob", 22) # __init__ được gọi tự động
print(s.name, s.age) # Bob 22
```

5 self trong Python là gì?

Trả lời:

self đại diện cho instance hiện tại của object. Nó được dùng để truy cập attribute và method của class.

```
class Person:
    def __init__(self, name):
        self.name = name # 'self' đại diện cho instance

    def get_name(self):
        return self.name

p = Person("John")
print(p.get_name()) # John
```

6 Bốn trụ cột (pillar) của OOP là gì?

Trả lời:

Bốn nguyên lý chính của OOP gồm:

- Encapsulation (đóng gói)
- Abstraction (trừu tượng hóa)
- Inheritance (kế thừa)
- Polymorphism (đa hình)



7 Encapsulation (đóng gói) là gì?

Trả lời:

Encapsulation là khái niệm gói dữ liệu (biến) và method lại với nhau, đồng thời hạn chế truy cập trực tiếp vào một số thành phần của object.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance # private attribute

    def deposit(self, amount):
        self.__balance += amount
    def get_balance(self):
        return self.__balance

a = Account(1000)
a.deposit(500)
print(a.get_balance()) # 1500
# print(a.__balance) # Lỗi (private)
```

Lưu ý: Dấu gạch dưới (_) khiến attribute trở thành private (name mangling).

8 Inheritance (kế thừa) trong Python là gì?

Trả lời:

Inheritance cho phép một class (con) kế thừa attribute và method từ một class khác (cha). Nó giúp tái sử dụng code.

```
class Parent:
    def show(self):
        return "This is parent"

class Child(Parent): # Child kế thừa từ Parent
    pass

c = Child()
print(c.show()) # This is parent
```

Lưu ý: Tái sử dụng code, dễ mở rộng, dễ bảo trì.

9 Các loại inheritance là gì?

Trả lời:

Python hỗ trợ các loại kế thừa sau:

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Single Inheritance	Multiple Inheritance
class A: pass class B(A): pass	class A: pass class B: pass class C(A, B): pass

10 Polymorphism (đa hình) trong Python là gì?

Trả lời:

Polymorphism cho phép các method thực hiện những hành vi khác nhau tùy vào object gọi chúng. Có thể đạt được thông qua method overriding và operator overloading.

```
class Animal:
    def sound(self):
        return "Some sound"

class Dog(Animal):
    def sound(self): # overridden method
        return "Bark"

d = Dog()
print(d.sound()) # Bark
```

QUICK FACTS

- ✓ Tên class nên viết hoa chữ cái đầu
- ✓ __init__() là constructor
- ✓ self đại diện cho instance hiện tại
- ✓ __ (double underscore) tạo private attribute

EXAMPLE CORNER

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hi, I am {self.name}"
```

KEY TAKEAWAYS

- ✓ OOP giúp viết code sạch, module hóa và dễ tái sử dụng
- ✓ Hiểu chắc 4 trụ cột của OOP là nền tảng quan trọng



9. EXCEPTIONS + FILES + MODULES

Date: / /



10 CÂU HỎI & TRẢ LỜI PHÒNG VẤN QUAN TRỌNG

A. EXCEPTIONS

1 Exception trong Python là gì?

Trả lời:

Exception là một sự kiện xảy ra trong quá trình chạy chương trình, làm gián đoạn luồng chạy bình thường.

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
print("Program continues...")
```

2 try, except, else, finally là gì?

Trả lời:

- try:** Khối code cần kiểm tra
- except:** Chạy khi có exception
- else:** Chạy nếu không có exception
- finally:** Luôn chạy (dọn dẹp - cleanup)

3 Một số exception phổ biến?

Trả lời:

Exception	Mô tả
ZeroDivisionError	Chia cho 0
ValueError	Giá trị không hợp lệ
TypeError	Sai kiểu dữ liệu
IndexError	Chỉ số ngoài phạm vi
KeyError	Key không có trong dict
FileNotFoundError	Không tìm thấy file
ImportError	Import module thất bại

4 Có thể dùng nhiều except cùng lúc không?

Trả lời:

Có, chúng ta có thể xử lý các exception khác nhau riêng biệt.

```
try:
    x = int(input("Enter number: "))
    y = 10 / x
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

5 Mục đích của việc raise một exception là gì?

Trả lời:

Chúng ta có thể chủ động phát sinh (raise) exception bằng từ khóa **raise**.

```
def check_age(age):
    if age < 18:
        raise ValueError("Age must be 18 or more")
check_age(16)
# ValueError: Age must be 18 or more
```

EXCEPTIONS BEST PRACTICES

- ✓ Xử lý exception cụ thể, tránh dùng except chung chung
- ✓ Dùng finally cho các thao tác dọn dẹp
- ✓ Không bỏ qua exception một cách âm thầm
- ✓ Raise thông báo lỗi rõ ràng, dễ hiểu
- ✓ Giữ khối try càng ngắn gọn càng tốt

B. FILES

1 Làm thế nào để mở một file trong Python?

Trả lời:

Chúng ta dùng hàm **open()**.

```
f = open("example.txt", "r") # đọc
f = open("example.txt", "w") # ghi
f = open("example.txt", "a") # thêm
f = open("example.txt", "r+") # đọc+ghi
```

Modes:

- r - Đọc (file phải tồn tại)
- w - Ghi (xóa file nếu)
- a - Thêm (nối vào cuối)
- r+ - Đọc và ghi
- x - Tạo file không được tồn tại
- b - Chế độ nhị phân (tên file)

2 Làm thế nào để đọc dữ liệu từ file?

Trả lời:

```
# Đọc toàn bộ file
f = open("file.txt", "r")
data = f.read()
print(data)
# Đọc từng dòng
f.seek(0)
for line in f:
    print(line.strip())
f.close()
```

3 Làm thế nào để ghi dữ liệu vào file?

Trả lời:

```
# Ghi (ghi đè)
f = open("file.txt", "w")
f.write("Hello Python!\n")
f.close()
# Thêm vào cuối
f = open("file.txt", "a")
f.write("\nKeep Practicing!")
f.close()
```

4 Sự khác nhau giữa read() và readline()?

Trả lời:

Method	Mô tả
read()	Đọc toàn bộ nội dung file
read(n)	Đọc n ký tự đầu tiên
readline()	Đọc một dòng mỗi lần
readlines()	Đọc tất cả dòng, trả về list

5 Câu lệnh with dùng cho file là gì?

Trả lời:

Dùng để quản lý tài nguyên tốt hơn (tự đóng/dung file).

```
with open("file.txt", "r") as f:
    data = f.read()
    print(data)
# File tự động đóng sau khối lệnh
```

FILE BEST PRACTICES

- ✓ Luôn đóng file (dùng with)
- ✓ Dùng đúng mode phù hợp
- ✓ Xử lý exception khi thao tác với file
- ✓ Tránh hardcoded đường dẫn file

C. MODULES

1 Module trong Python là gì?

Trả lời:

Module là một file chứa các định nghĩa và câu lệnh Python. Nó giúp code có tính tổ chức và tái sử dụng.

Ví dụ: **math.py**

```
# math.py
def add(a, b):
    return a + b
```

2 Làm thế nào để import một module?

Trả lời:

```
import math
import math as m
from math import sqrt
from math import *

print(math.pi)
print(m.sqrt(16))
print(sqrt(25))
```

3 Sự khác nhau giữa import và from import?

Trả lời:

import module	from module import
Import toàn bộ module	Import hàm/biến cụ thể
Truy cập bằng module.n	Truy cập trực tiếp
math.sqrt(16)	sqrt(16)

4 __name__ trong Python là gì?

Trả lời:

Là một biến đặc biệt có giá trị **"__main__"** khi script được chạy trực tiếp.

```
if __name__ == "__main__":
    print("Program is run directly")
```

5 Làm thế nào để tạo module của riêng bạn?

Trả lời:

Tạo một file **.py**, viết các hàm trong đó và import nó.

```
# mymodule.py
def greet():
    return "Hello!"
```



10. BÀI TOÁN CODE + CHEAT SHEET PHÒNG VẤN

Date: / /



10 BÀI TOÁN CODE QUAN TRỌNG + CHEAT SHEET PHÒNG VẤN ĐẦY ĐỦ

1. Two Sum

Cho một list số và một target, trả về chỉ số của hai số có tổng bằng target.
Input: nums=[2,7,11,15], target=9
Output: [0, 1]

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        if target - num in seen:
            return [seen[target-num], i]
        seen[num] = i
```

2. Palindrome Number

Kiểm tra một số có phải palindrome (đọc xuôi ngược) không.
Input: x=121 - True
Input: x=121 - False

```
def is_palindrome(x):
    if x < 0:
        return False
    original = x
    rev = 0
    while x:
        rev = rev*10 + x%10
        x //= 10
    return original == rev
```

3. FizzBuzz

In số từ 1 đến n. Fizz nếu chia hết cho 3, Buzz nếu chia hết cho 5, FizzBuzz nếu cả hai.
Input: n=15
Output: 1 2 Fizz 4 Buzz...

```
def fizz_buzz(n):
    for i in range(1, n+1):
        if i%3==0 and i%5==0:
            print('FizzBuzz')
        elif i%3==0:
            print('Fizz')
        elif i%5==0:
            print('Buzz')
        else:
            print(i)
```

4. Reverse a String

Đảo ngược một chuỗi cho trước.
Input: s="hello"
Output: "olleh"

```
def reverse_string(s):
    return s[::-1]
```

5. Find Maximum in Array

Tìm phần tử lớn nhất trong một mảng.
Input: arr=[3,1,7,4,9,2]
Output: 9

```
def find_max(arr):
    max_val = arr[0]
    for num in arr:
        if num > max_val:
            max_val = num
    return max_val
```

6. Count Vowels

Đếm số nguyên âm (vowels) trong một chuỗi.
Input: s="programming"
Output: 3

```
def count_vowels(s):
    vowels = 'aeiouAEIOU'
    count = 0
    for ch in s:
        if ch in vowels:
            count += 1
    return count
```

7. Fibonacci Series

In n số Fibonacci đầu tiên.
Input: n=7
Output: [0,1,1,2,3,5,8]

```
def fibonacci(n):
    fib = [0, 1]
    for _ in range(2, n):
        fib.append(fib[-1]+fib[-2])
    return fib[:n]
```

8. Remove Duplicates

Xóa phần tử trùng lặp trong list, giữ nguyên thứ tự.
Input: [1,2,2,3,4,4,5]
Output: [1,2,3,4,5]

```
def remove_duplicates(nums):
    seen = set()
    result = []
    for num in nums:
        if num not in seen:
            seen.add(num)
            result.append(num)
    return result
```

9. Factorial of a Number

Tính giai thừa (factorial) của một số n.
Input: n=5
Output: 120

```
def factorial(n):
    if n < 0:
        return "Underflow"
    res = 1
    for i in range(1, n+1):
        res *= i
    return res
```

10. Check Prime Number

Kiểm tra một số có phải số nguyên tố không.
Input: n=29
Output: True

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True
```

★ PYTHON INTERVIEW CHEAT SHEET ★

1. PYTHON BASICS

Python là ngôn ngữ bậc cao, thông dịch, đa năng.
Thụt lề (indentation) rất quan trọng.
Mọi thứ trong Python đều là object.
Comment một dòng
""" Comment nhiều dòng """

5. DATA STRUCTURES

List: Có thứ tự, mutable, cho phép trùng lặp.
Tuple: Có thứ tự, immutable.
Set: Không thứ tự, phần tử duy nhất.
Dictionary: Cặp Key-Value, mutable.
Common Methods:
List: append(), pop(), remove(), sort(), len()
Tuple: count(), index(), len()
Set: add(), remove(), union(), intersection()
Dict: keys(), values(), items(), get(), update()

9. FILE HANDLING

Mode:
r - Đọc (mặc định)
w - Ghi (ghi đè)
a - Thêm vào cuối
r+ - Đọc & Ghi
w+ - Ghi & Đọc

```
with open("file.txt", "r") as f:
    data = f.read()
```

2. DATA TYPES

Type	Value
int	10, -5, 100
float	3.14, -0.01
str	"hello", 'python'
bool	True, False
list	[1, 2, 3]
tuple	(1, 2, 3)
set	{1, 2, 3}
dict	{'a': 1, 'b': 2}

6. FUNCTIONS

Dùng def để tạo function.
*args - positional argument số lượng bất kỳ.
**kwargs - keyword argument số lượng bất kỳ.
Lambda - function không tên (anonymous).

```
def add(a, b=0):
    return a + b
lambda x: x * 2
```

10. MODULES & PACKAGES

Module - một file .py
Package - thư mục có __init__.py
Import module bằng import

```
import math
from math import sqrt
import math as m
```

3. OPERATORS

Arithmetic: +, -, *, /, %
Comparison: ==, !=, >, <, >=, <=
Logical: and, or, not
Bitwise: &, |, ~, ^, <<, >>
Assignment: =, +=, -=, *=, /=, %=

7. OOP

Class - khuôn mẫu (blueprint)
Object - instance
__init__() - constructor
Inheritance, Polymorphism, Encapsulation, Abstraction

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi, I am {self.name}"
```

11. BUILT-IN FUNCTIONS

print(), len(), type(), input(), int(), float(), str(), bool(), max(), min(), sum(), abs(), round(), sorted(), range(), enumerate(), zip()

14. USEFUL TIPS

✓ Dùng list/dict/set comprehension.
✓ Dùng enumerate() thay vì range(len()).
✓ Dùng in để kiểm tra thành viên (membership).
✓ Ưu tiên defaultdict, Counter từ collections.
✓ Dùng with statement khi thao tác tập file.
✓ Giữ code Pythonic & dễ đọc.

4. CONTROL FLOW

if else
for item in iterable: #block
while condition: #block
break, continue, pass

8. EXCEPTIONS

Dùng try, except, else, finally

```
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print("Error:", e)
else:
    print("No Error")
finally:
    print("Done")
```

12. COMMON SPECIAL METHODS

Method	Chức năng
__init__(self)	Constructor
__str__(self)	Biểu diễn chuỗi
__repr__(self)	Biểu diễn chính thức
__len__(self)	Độ dài object
__eq__(self, other)	Kiểm tra bằng nhau
__gt__(self, other)	So sánh lớn hơn

13. TIME COMPLEXITY GUIDE

Kí hiệu	Tên gọi	Ví dụ
O(1)	Hằng số	Truy cập mảng
O(log n)	Logarit	Binary Search
O(n)	Tuyến tính	Linear Search
O(n log n)	N-log-N	Merge Sort
O(n^2)	Bậc hai	Nested Loops
O(2^n)	Hàm mũ	Recursive (fib naive)
O(n!)	Giai thừa	TSP (brute force)